A Bayesian Optimisation Approach to Inventory Management in the Bristol Stock Exchange

Kaloyan Kirov

Department of Physics, University of Bath, Bath BA2 7AY, United Kingdom
E-mail: kk2085@bath.ac.uk

Abstract

This study explored whether a purpose-designed algorithmic trader, with its parameters tuned through Bayesian optimisation, could compete profitably against established strategies in the Bristol Stock Exchange (BSE) simulator. A preliminary analysis of six baseline traders revealed that inventory accumulation and trade-to-trade adaptation, rather than learning sophistication, were the primary drivers of profitability in this market environment. This motivated the design of a Custom Trader with inventory management controls, whose three defining parameters were optimised using a Gaussian Process over 100 iterations. The optimised trader was then tested against all six baselines across 150 independent trials of four market conditions, as was done in the preliminary study. It finished second overall by mean profit at 5,165, behind ZIP at 7,061, and led the field outright in the shock-up condition. It was the only trader to return a positive profit in more than 90% of trials across all four conditions, and achieved the highest ratio of mean profit to standard deviation of any trader in the field, consolidating its consistency. Bootstrap resampling over 10,000 iterations confirmed that the ranking is statistically valid and not a product of sampling variation across the trials.

Introduction

An exchange of goods between parties is a practice that has existed as long as the human species has. Items such as crops, materials and money have always been the drivers in exchanges, where as long as you had goods, you could negotiate with another party and trade for their goods. In more modern times, financial markets have been environments where the exchange of money for a small ownership, or shares, of a company or product has seen large change, through vast developments in technology, economic theory and mathematics.

The beauty of these markets is that anyone can choose to get involved in the buying or selling of shares, in the hopes of coming out of it with a profit. However, there are many factors that complicate the goal of profiting from your trades, such as knowing what price the commodity you just bought or sold will be in x seconds, which all comes down to how other traders alongside you in the market act. This could be due to external factors, such as a company CEO being found guilty of fraud, or a company CEO publishing the company's incredible annual returns, but it would be disingenuous to tell you that trading in these markets would be that simple. In the present day, major stock exchanges such as the New York Stock Exchange alone report average daily trading volumes exceeding one billion shares [1], which create just as many price changes of the stock you are trading per day. The result of humans struggling to keep up with this constant flow of information has led to the development of algorithmic trading: the use of computer programs to execute financial trades automatically through pre-set instructions, led by variables such as the time evolution of price, volume and volatility of the commodity being traded.

Hundreds of millions have been poured by individual banks into the research and talent it takes to build the most complex and profitable algorithms [2], which can sound very intimidating for someone who just wanted to trade a stock because they heard they can quit their 9-5

if they are good enough. Fortunately, there are resources and platforms that allow anyone to build a trading bot with their own strategies, and test them against pre-made algorithms, varying in complexity, before choosing to step into a real, digital financial exchange. An excellent example, that was used in this research, is the Bristol Stock Exchange (BSE).

At the core, the BSE is a computational framework that allows a user to simulate a financial market, for use in university teaching and research [3]. The open-sourced Python tool provides a virtual financial exchange environment, where transactions occur through a Limit Order Book (LOB), in which all outstanding buy orders (bids) and sell orders (asks) are logged. The trades occur via a Continuous Double Auction (CDA): the buyer increases their offer and the seller decreases their offer until they meet at an agreed price. The LOB will take in every order and execute the trade consisting of the user with the highest bid (best bid) and the user with the lowest ask (best ask). Every order submitted to the LOB carries a limit price, which is the worst price a trader is willing to accept. A buyer's limit price is the maximum they are prepared to pay, and a seller's limit price is the minimum they are prepared to accept. The LOB matches orders when the best bid meets or exceeds the best ask, and no trade occurs until that condition is satisfied. When a trade does execute, the profit a trader makes on that transaction is the difference between the agreed price and their limit price, since the limit price represents the worst outcome they were willing to accept and anything better than that is gain. A trader that consistently executes trades at prices better than their limit price is said to have a positive profit margin, defined as the fractional difference between their quoted price and their limit price. The price of a tradeable item within a financial market in the BSE is dictated by comparing the supply and demand of that item, such that the trading activity at time step t directly influences the price at time step $t+1$. This process is, of course, put into motion by trading algorithms, which have built-in strategies that dictate how they trade within this simulated environment.

The traders available in the BSE span a wide range of strategies, from trivially simple to algorithms that are the foundations of decades of research in market microstructure. The six baseline traders used in this study are:

- Giveaway (GVWY) quotes exactly at its limit price, accepting zero profit margin on every trade in order to guarantee execution.
- Zero-Intelligence Constrained (ZIC) [4] submits random quotes drawn uniformly between the limit price and the best opposing quote, making no attempt to learn or optimise.
- Zero-Intelligence Plus (ZIP) [5] develops ZIC by equipping each trader with an adaptive profit margin μ, which it updates after every observed transaction in a market session. When a trade executes at price p, ZIP computes a target price τ by perturbing p upward or downward depending on whether it needs to widen or tighten its margin. The margin is then updated by blending a correction based on the gap between the current quote and the target with a fraction of the previous update, so that successive adjustments do not change direction too abruptly:

$$\Delta_t = (1-\gamma)(\beta(\tau - q_t)) + \gamma\Delta_{t-1} \tag{1}$$

$$\mu_{t+1} = \frac{q_t + \Delta_t}{l} - 1 \tag{2}$$

  where q_t is the current quoted price, l is the limit price, β controls how strongly the margin responds to the gap between the current quote and the target, and γ smooths successive updates by blending the new correction with the previous one. Over many time steps, ZIP converges on a margin that reflects the competitive level of the market.

- Shaver (SHVR) undercuts the best ask by one unit of currency (tick) when selling, by putting through an ask at 99 when the previous best ask was 100, and overcuts the best bid by one tick when buying, securing queue priority without a learning mechanism.
- Sniper (SNPR) withholds all orders until the final 20% of the trading session, attempting to exploit end-of-session price convergence, where it operates with a modified SHVR strategy, undercutting and over-cutting asks and bids by a larger amount of ticks, at the cost of a low overall participation rate.
- Parameterised-Response Stochastic Hill-Climber (PRSH) [6] enters a market session by maintaining 4 parallel strategies internally, each defined by an aggressiveness parameter $s \in [-1, +1]$, where negative values produce conservative quotes far from the best opposing price and positive values produce aggressive quotes close to it. Only one strategy is active at any time, so each PRSH trader submits orders in their given time frame of $\texttt{strat\_wait\_time} = 60$ time steps. After the 60 time steps, PRSH switches to the next strategy and by the end of this 240 time step cycle, it scores the strategies by profit earned while they were active, keeping the highest-scoring strategy unchanged for the next cycle, and updating the remaining 3 strategies by shifting their s values toward the winner's s value by a small random amount. Over repeated evaluation cycles this pulls all strategies toward whichever aggressiveness level has been most profitable, concentrating the population around a locally optimal value of s. The 60-step evaluation window means PRSH can only adapt roughly 15 times across a session of 3600 time steps.

In recent years, algorithmic trading has been estimated to account for up to 80% of activity in major financial markets [7]. The strategies behind these algorithms span a wide range of complexity, from simple real-time adaptation to approaches rooted in statistical learning and optimisation; this is no different in some of the strategies in the BSE. What many of these strategies share, however, is a dependence on parameters whose optimal values cannot be derived analytically, such that the only way to know whether a given parameter combination works is to run the algorithm and observe the result. This makes conventional tuning approaches such as grid searching computationally expensive, as they evaluate the parameter space exhaustively without using prior results to guide where to look next. Bayesian optimisation addresses this directly by building a probabilistic model of the objective function and using it to decide where to evaluate next, concentrating effort in regions most likely to yield improvement [8]. It is therefore well-suited to algorithmic trader design, where each evaluation requires a full market simulation.

Therefore, this report will aim to explore and evaluate the integration of a new algorithmic trader, built on Bayesian optimisation in order to maximise profitability. Financial markets of varying conditions were simulated within a modified BSE environment, and the behaviour and interactions of all traders were assessed across those conditions, through which a thorough analysis of the outcomes was then used to identify the key drivers of profitability, and to evaluate how the new trader performs against the established field.

Method

Before running any simulations that yield analysis that dictate the strategy of the Custom Trader (CT), it was necessary to establish if the way profit is measured in the BSE was appropriate for this study. The profit definition previously mentioned can be imagined as a customer wanting a trader to buy a stock at £5.00 and the trader buying it at £4.50, therefore keeping the £0.50 as

Trader Type	Profit	No. of Trades
SHVR	604.65	48.0
PRSH	590.57	39.3
GVWY	507.24	51.6
⋮	⋮	⋮

Table 1: The results of 100 independent trials with 3600 time steps with 6 of the baseline trader types, displaying the mean profit per trader type and number of trades executed by trader type. This was simulated using Cliff's intended market environment, with assigned buyers and sellers, as well as a dynamic evolution of an equilibrium market price through supply and demand intersection.

"profit". Accompanying this with the fact that the traders are built in such a way that their quotes will never exceed the transaction price in either way means that there would be no "losses" by any trader in a market session. The flaws of this system can be put on display by running simulations of simple market sessions, within Cliff's BSE framework [3] that allowed the traders to establish equilibrium pricing through an evolving supply and demand curve, which yielded the results shown in Table 1, which shows that a strategy as simple as GVWY or SHVR that maximise trading frequency, with no consideration or evaluation of performance, can be the most successful; a not very realistic approach to evaluating performance against more strategies tested in more complex environments. Therefore, it was worth reconsidering the way profit was measured to perform further notable analysis, which would aim to reduce the constraints BSE imposes (others will be discussed later) compared to real financial markets.

This is done through introducing a new profit measure, P, defined as

$$P = C_f + (I \times \overline{SP}) - C_0, \tag{3}$$

where C_f is the final balance of the trader, I is the inventory of the trader i.e. number of shares owned at a given time step, $\overline{SP}$ is the mean share price of the final 50 transactions in the session at a given time step and C_0 is the initial cash balance of 10,000 that all traders start off with. The new implementation of an inventory for each trader allows for losses, as opposed to profits, punishing the traders that have a large inventory in either way (a positive 'long' position or a negative[1] 'short' position), or benefiting in other cases. Alongside this, it was important to remove the distinction between 'buyers' and 'sellers' within the BSE in order to be consistent with the idea that inventory management was an important feature to consider for CT, and be a metric that isn't just predetermined by the role a trader is assigned.

Upon doing this, market sessions were simulated, with a foundation that was kept constant for every simulation for the rest of the research done. Each session lasted 3600 time steps, with 150 independent trials run per condition. Each session contained 5 traders of each type, giving a total of 30 traders per session in the preliminary study and 35 in the main simulation once CT was introduced. The base price was manually fixed at $SP_0 = 100$, representing the base market price around which all trading activity occurs. Four conditions were defined, each distinguished by an offset in the base price, in order to understand how the traders in the market behave with sudden changes applied to SP_0:

[1] Acquiring a negative number of shares owned is plausible as long as it's defined correctly. In this case, it means taking a 'short' position, which offers huge benefits in cases where the price of a share decreases, as you can buy back the shares you owe at a price cheaper than when you were selling shares, resulting in having more money than what you started with, a profit. This is best shown in *The Big Short (2015)*, which documented traders that were able to make substantial profits in financial markets following the 2008 Financial Crisis.

- **Stable:** No offset is applied to the base price, serving as the baseline condition.
- **Shock up:** A +30 applied at t = 1200, holding the market price at a base of 130 for the remainder of the session.
- **Shock down:** A -30 applied at t = 1200, holding the market price at a base of 70 for the remainder of the session.
- **Multi-regime:** The session is divided into ten regimes of 360 time steps. For each regime after the first, a multiplier $m \sim \mathcal{N}(1.0,\ 0.2^2)$, is drawn independently, giving an offset of $(m - 1.0) \times 100$ in each regime.

At each time step, every trader is designated randomly as either a buyer or seller with equal probability. Buyers receive a limit price ℓ drawn uniformly from $[100,\ 120]$ and sellers from $[80,\ 100]$, expressed as

$$\ell \sim \begin{cases} \mathcal{U}(100,\ 120) & \text{for bids} \\ \mathcal{U}(80,\ 100) & \text{for asks.} \end{cases} \tag{4}$$

This ensures that buyers always receive limit prices at or above the equilibrium price and sellers always receive limit prices at or below it, creating a natural overlap around SP_0 within which trades can occur. It's important to note that ℓ shown above is for the case of a 'Stable' market, and changes in accordance with the market condition, such as for a buyer in the shock-up condition receiving $\ell \sim \mathcal{U}(130, 150)$, and a seller receiving $\ell \sim \mathcal{U}(100,\ 130)$.

The above-mentioned was implemented by extending Cliff's framework [3], rather than replacing it, so that the core exchange mechanics of the LOB and CDA remain unchanged. Each trader type was extended to track an inventory counter that increases by one for each purchase and decreases by one for each sale, with profit computed at each time step using Equation 3. CT was implemented as a separate subclass built on top of the trader base, with its defined parameters running alongside the exchange mechanics.

The preliminary study was conducted with the goal to lay the groundwork for what CT needed to prioritise by revealing what strategies can make the most profit when averaging out the profits from each trial in each condition. The results from this simulation revealed two consistent drivers of profitability in this market structure, which were the speed at which a trader adapts its strategy and the inventory position it accumulates over the course of a session. ZIP's trade-to-trade margin adjustment allows it to track the market continuously, and its resulting short position proves consistently profitable under the construction of profit in Equation 3, as the cash it accumulates from selling units at or above the equilibrium price more than offsets the cost of its negative inventory at session end. PRSH, by contrast, evaluates strategies 15 times for session lengths of 3600, meaning it can't adapt as quickly as a trader like ZIP, limiting its ability to learn and take advantage of evolving market prices the way a desired strategy would aim to do.

The observations directly motivated the design priorities of CT and were quantised through three parameters that were built on the modifications to the BSE environment and therefore defined its behaviour:

- **Learning rate**, α: CT maintains a profit margin μ and quoted price q defined in the same way as ZIP's, but uses a simpler update rule without the smoothing term, controlled by a single learning rate α. If CT is assigned a limit price of 100, the minimum of the buyer range $\mathcal{U}(100,\ 120)$ in a stable market, and applies a margin of 0.05, it submits a bid

of $100 \times (1 - 0.05) = 95$. The margin is updated after every transaction that occurs in the market. CT observes every transaction through the LOB, which reveals each trade price to all traders after execution, meaning CT updates its margin in response to market activity regardless of whether it was involved in that trade. When a transaction executes at price p, CT compares p to its own most recent quote q. If CT was bidding and $p \leq q$, a trade could have executed at those prices, so the margin is widened to become more profit hungry:

$$\mu_{t+1} = \mu_t \cdot (1 + \alpha) \tag{5}$$

If $p > q$, the margin was too conservative and is tightened:

$$\mu_{t+1} = \mu_t \cdot (1 - \alpha) \tag{6}$$

with symmetric logic applied when CT is posting an ask. The margin is clipped to $[0.01,\ 0.50]$ after each update to prevent extreme quoting behaviour in either direction. A lower α produces slower, more stable adaptation, while a higher α causes faster but more erratic responses to market activity.

- **Position Limit**, L: An unchecked inventory position can erode profit regardless of how well a trader performs on individual trades. CT therefore tracks its current inventory at all times and applies two complementary mechanisms to manage it. Once $|I|$ exceeds half the position limit, a bias term is added to the current margin:

$$\text{bias} = -0.02 \times \frac{I}{L} \tag{7}$$

which nudges CT's quotes toward $I = 0$ before the hard limit is reached. For example, if $L = 200$ and $I = 190$, Equation 7 gives a bias of -0.019. If the margin at that point was 0.99, it becomes 0.971, reducing CT's likelihood of executing a buy order and increasing the likelihood of executing a sell, pulling the inventory back toward zero. The same logic applies in reverse for a large negative inventory. If $|I|$ reaches L itself, CT enters *defensive mode*, in which the effective margin is floored at 0.15, a dimensionless value on the same scale as μ, so that:

$$\mu_{t+1} = \max(\mu_t,\ 0.15) \tag{8}$$

meaning CT's quoted prices move further from the limit price and its orders become less competitive and therefore less likely to execute. A higher position limit allows CT to accumulate more inventory before triggering this response.

- **Volatility Threshold**, σ_v: During volatile periods, the market becomes more unpredictable and the risk of accumulating a harmful inventory position increases. Volatility here is measured as the standard deviation of the 50 most recent transaction prices observed in the session, giving a rolling estimate of how much prices are fluctuating around their recent average:

$$\hat{\sigma} = \text{std}(p_{t-49},\ p_{t-48},\ \ldots,\ p_t) \tag{9}$$

where p_t is the price of the most recent transaction at time step t. If $\hat{\sigma}$ exceeds σ_v, CT enters defensive mode through this second route, independent of its current inventory level, applying the same floored margin described above. CT exits defensive mode only once $\hat{\sigma}$ falls below $0.5\sigma_v$, requiring conditions to improve before normal trading resumes.

All three parameters had to take a specific value, and the combination of parameter values,

$$\mathbf{x} = (\alpha,\ L,\ \sigma_v), \tag{10}$$

that can be used for CT to maximise its profit was evaluated by Bayesian optimisation. The search was conducted over the ranges $\alpha \in [0.001,\ 0.30]$, $L \in [20,\ 800]$, and $\sigma_v \in [3.0,\ 15.0]$. The upper bound on L was set generously to allow CT to approach inventory values similar to and beyond ZIP to see if any additional profit could be discovered, while the lower bound on α was set well below ZIP's typical learning rate to allow for slower adaptation. The bounds on σ_v were chosen to span the range between stable market conditions, where volatility is low, and the most volatile multi-regime sessions observed in the preliminary study.

The optimal parameter combination for CT, defined as the $\mathbf{x}$ that produces the highest profits, could not be derived analytically. Therefore, Bayesian optimisation was used to search the parameter space efficiently, as it is a technique with an established track record in tuning the parameters of computationally expensive black-box functions in a financial trading context [9]. Here, the black box is the mean profit of CT across a set of simulations, as the only way to evaluate how profitable the parameters of $\mathbf{x}$ are is to run CT against the other traders and record the result. There is no formula that produces the profit directly from the parameters, as the outcome depends on the stochastic interactions of 35 traders across 3600 time steps. Bayesian optimisation searches this space more efficiently than "brute-force" approaches such as grid searching, by using each evaluation to inform where to look next rather than testing parameter combinations exhaustively.

The procedure works by building a probabilistic model of the objective function and using that model to decide where to evaluate next, rather than searching blindly. The model used here is a Gaussian Process (GP), which at any point $\mathbf{x}$ in the parameter space maintains a probability distribution over what the profit at that point might be:

$$f(\mathbf{x}) \sim \mathcal{N}\left(\mu(\mathbf{x}),\ \sigma^2(\mathbf{x})\right) \tag{11}$$

where $\mu(\mathbf{x})$ is the GP's best estimate of the profit at $\mathbf{x}$, and $\sigma^2(\mathbf{x})$ is the associated uncertainty. The GP was initially built from 20 random evaluations, each assigning CT a random parameter combination in $\mathbf{x}$, simulating 5 CT traders with this $\mathbf{x}$ against 5 of the other trader types each, in 10 sessions per condition, and recording the mean profit observed for this value of $\mathbf{x}$. These initial points give the GP a rough picture of how profitable different regions of the parameter space are. The mathematical expressions for $\mu(\mathbf{x}^*)$ and $\sigma^2(\mathbf{x}^*)$, derived from the Matérn covariance kernel, are given in the Appendix.

Once the first 20 evaluations were complete, the objective of the GP was to use this information to decide where to evaluate next. Evaluating only where the mean is highest risked missing better regions that have not yet been explored and evaluating only where uncertainty is high wasted evaluations in regions that may have no chance of improving on the current best result. This trade-off between "exploitation" and "exploration" was resolved by an acquisition function, Expected Improvement (EI) [10] in this case, which scored each trial $\mathbf{x}$ by how much improvement over the current best profit it was expected to deliver, accounting for both $\mu(\mathbf{x})$ and $\sigma^2(\mathbf{x})$. At each of the remaining 80 GP iterations, EI selected the next parameter vector to evaluate, from which CT was then run under those parameters, and the result was fed back into the GP to sharpen its model. The Bayesian optimisation was implemented using the Scikit-Optimize library [11], which provides a Gaussian Process minimiser with Expected Improvement as the acquisition function. The full Python implementation is given in the Appendix. Upon receiving the optimal parameters in $\mathbf{x}$, which determined the combination that

yielded the highest mean profit for CT, it was ready to go up against the rest of the traders in an environment identical to the one used in the preliminary study.

Ideally, CT would become the most profitable strategy in the market, but it was important to recognise that a single mean profit figure across 150 trials was not enough to say with confidence that CT, as a trading algorithm, is "better" that the rest (or most, or none) of the traders in the BSE. Any trader could get lucky, particularly over the finite sessions run for this research. Therefore, it was important to include a metric or test that indicated CT's consistency in outperforming other strategies, as well as being an effective strategy in the long-term.

In an attempt to ensure this was the case, bootstrap resampling was applied to the profit distributions of each trader type, where it treats the 150 observed trial profits for each condition as a representative sample of all possible outcomes, and simulates the sampling process directly. Let $X = \{x_1, x_2, \ldots, x_N\}$ represent the original sample of $N = 150$ observed trial profits for a given trader. For each bootstrap iteration k, a resample $X^{*(k)} = \{x_1^{*(k)}, x_2^{*(k)}, \ldots, x_N^{*(k)}\}$ is constructed by randomly sampling N values from X with replacement, meaning each value is returned to the pool after selection so that any single observation can appear multiple times in the resample while others may not appear at all. This produces a new sample of the same size as the original but with a different composition, simulating what another set of 150 trials might have looked like. The mean of this resample is then computed as:

$$\bar{x}^{*(k)} = \frac{1}{N} \sum_{i=1}^{N} x_i^{*(k)} \tag{12}$$

After $K = 10{,}000$ iterations, this yields a distribution of bootstrapped means $\{\bar{x}^{*(1)}, \bar{x}^{*(2)}, \ldots, \bar{x}^{*(K)}\}$, from which the 95% confidence interval is taken as the 2.5th and 97.5th percentiles of this distribution:

$$\mathrm{CI}_{95} = \left[\bar{x}^{*}_{(0.025)}, \bar{x}^{*}_{(0.975)}\right] \tag{13}$$

If the confidence intervals of two traders do not overlap, it can be stated with high confidence that their difference in mean profit is not by chance.

To assess long-term performance, a cumulative profit analysis was also done. For each of $B = 1000$ bootstrap iterations, a resample $X^{*(k)}$ of $M = 500$ values is drawn with replacement from the 150 observed profits, representing 500 simulated future sessions. The cumulative profit after session j in iteration k is:

$$C_j^{*(k)} = \sum_{i=1}^{j} x_i^{*(k)}, \qquad j = 1, \ldots, M \tag{14}$$

After B iterations, the mean cumulative trajectory and its 95% confidence interval at session j are:

$$\bar{C}_j^{*} = \frac{1}{B} \sum_{k=1}^{B} C_j^{*(k)}, \qquad \mathrm{CI}_{95}(j) = \left[\bar{C}^{*}_{j,(0.025)}, \bar{C}^{*}_{j,(0.975)}\right] \tag{15}$$

Since successive sessions are independent, the expected cumulative profit grows linearly as $\mathbb{E}[C_j^{*}] = j\,\bar{x}$, where $\bar{x}$ is the sample mean profit per session. The width of the confidence band grows as $\sqrt{j}$, reflecting the accumulation of uncertainty over a longer horizon, which means that a trader with a consistent mean profit and a narrow band is more reliable than one with the same mean but a wide, unpredictable spread.

Trader Type	Profit	No. of Trades
ZIP	10,053	3,636
PRSH	2,915	1,189
ZIC	1,840	578
SNPR	-1,056	1,213
SHVR	-5,379	5,433
GVWY	-8,176	6,277

Table 2: The results of 150 trials per condition, for all 4 conditions, with the 6 baseline traders present (5 traders per type) and going against each other. Each session lasted 3600 time steps, from which the mean profit per trader and total number of trades could be extracted.

Results

Preliminary Findings

As shown in Table 2, ZIP leads the field in profitability with a mean profit of 10,053 and the highest trade count of the three profitable trader types, with the 5 ZIP traders collectively executing 3,636 trades on average across all conditions. PRSH and ZIC follow with positive mean profits of 2,915 and 1,840 respectively. SNPR, SHVR, and GVWY are consistently unprofitable, with GVWY recording the worst mean profit of all six strategies at $-8{,}176$ despite executing 6,277 trades on average, the highest trade count in the field. This was a direct consequence of the redefined profit metric and the stochastic nature of limit price assignment. The mechanism behind these results lies in how each trader's strategy causes it to accumulate inventory over the course of a session. ZIP's margin adaptation rule tightens its selling margin after observing a trade, which causes it to post aggressive asks close to the best ask price throughout the session. The result is that ZIP sells more units than it buys, building a short position of approximately -270 shares by $t = 3600$. Under the symmetric price schedule centred at $SP_0 = 100$ this is consistently profitable, as the cash accumulated from selling units at or above equilibrium more than offsets the cost of closing the short position at the session's end. PRSH's behaviour is structurally opposite, as its hill-climbing strategy converges toward more aggressive parameter values $s \in [0.7,\ 1.0]$ in stable conditions. This means that its traders' buy orders execute more readily than sell orders, so PRSH drifts toward a large long position of roughly 400 to 600 shares by the session's end. Its profit therefore depends on the price remaining at or above the average price at which those shares were acquired.

This inventory dependence becomes a direct vulnerability when market conditions change. Figure 1 shows that ZIP's short position, which seemed to be profitable in stable conditions, becomes a liability in the shock-up condition, as the sudden price increase at $t = 1200$ raises the losses due to the $(I \times \overline{SP})$ term from Equation 3 increasing, producing a pronounced drop in mean profit. ZIP also performs considerably worse in the multi-regime condition, where repeated random offset changes drive price swings that repeatedly expose this short position. These findings directly motivated the inclusion of the position limit L and volatility threshold σ_v in CT's design, providing explicit mechanisms to contain inventory accumulation and reduce exposure during periods of high market volatility.

Bayesian Optimisation

Figure 2 shows the parameter space explored by the GP and the convergence of the best observed mean profit over the 100 iterations. Through the GP's search, the optimal parameters

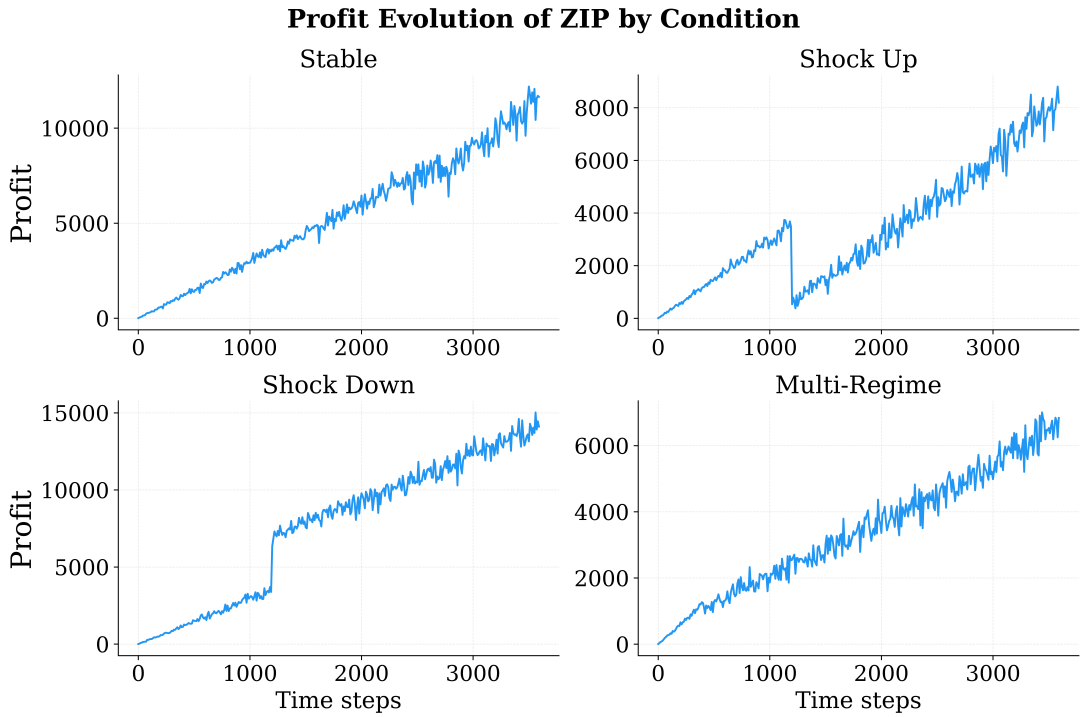

Figure 1: Mean profit evolution per ZIP trader across all four market conditions in the preliminary simulations, averaged over 150 trials.

were found to be

$$\mathbf{x} = (0.0452, 294, 14.987). \tag{16}$$

These parameter values resulted in a profit of 4,971, averaged over 40 sessions across all conditions. The 3D plot shows how the GP evolved in its search of the optimal $\mathbf{x}$ over every iteration. The GP covered a wide range of L and α, with a consistent preference for $\sigma_v \geq 8$ throughout its search. The optimal $\sigma_v = 14.99$ sits exactly at the upper boundary of the search range $[3.0,\ 15.0]$, indicating that within the space explored, no lower threshold consistently improved performance. This helped to show that CT was penalised for going into defensive mode too often in sessions where there was more room for improvement and hinted at the idea that increasing the parameter space would yield higher mean profits.

Custom Trader Performance

Table 3 shows the results of the main simulation, in which all trader types competed across all conditions. CT finished second overall by mean profit at 5,165, trailing ZIP (7,061) but outperforming PRSH (1,951) in third. Alongside mean profit, an informative measure is the ratio of mean profit to standard deviation of the profits, σ, obtained per type, which helps to quantify how much profit a trader earns per unit of variability in its returns. It follows that a higher ratio shows that, for a trader type, the mean is achieved consistently rather than through a small number of high-variance outcomes. As shown by this measure, CT leads the field at 2.50, compared to ZIP's 1.50 and PRSH's 0.31. CT's standard deviation of 2,064 is less than half of ZIP's 4,723, while CT also executes more trades per session than any other profitable trader at 4,675, compared to ZIP's 3,750 and PRSH's 1,212.

Figure3 shows the mean inventory trajectories of CT, ZIP and PRSH across all four conditions throughout a session. The structural difference in inventory management between the three traders is important to initially establish, in order to later justify the consequences of their

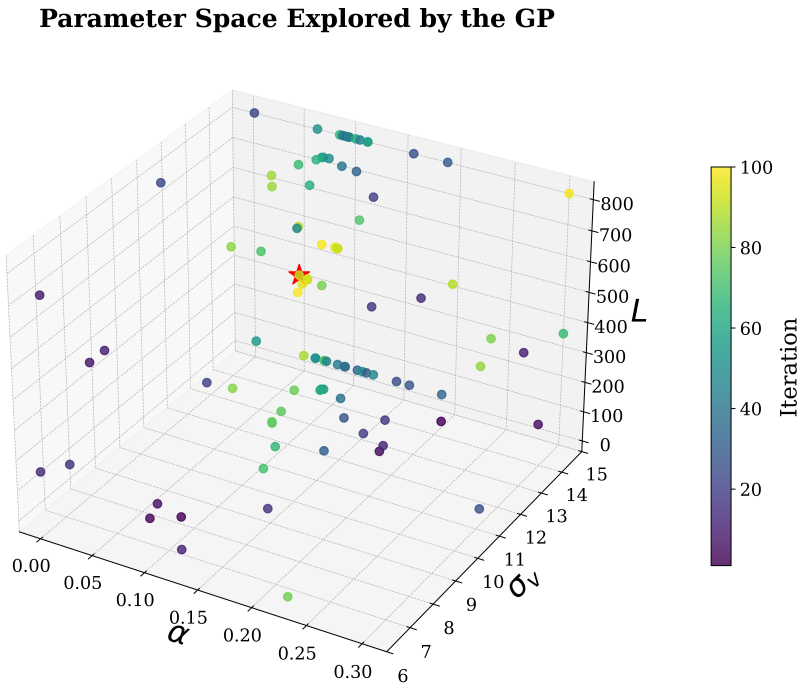

Figure 2: Parameter space explored by the Gaussian Process over 100 iterations, with axes showing α, L and σ_v. Each point represents one evaluated parameter combination, coloured by iteration number. The star marks the optimal combination $\mathbf{x} = (0.0452,\ 294,\ 14.987)$.

inventory accumulation. ZIP drifts into a steadily growing short position in every condition, reaching approximately -450 shares by $t = 3600$ in stable and shock-down runs. PRSH moves in the opposite direction, accumulating a long position of 450 to 550 shares by session end across most conditions. CT, by contrast, remains close to zero throughout, displaying the position limit and bias mechanism working together to resist accumulation in either direction.
Figure 4 shows the distribution of per-trial profits across all four conditions. It shows that CT is profitable in all 150 trials in the stable, shock-up, and shock-down conditions, and in 93.2% of multi-regime trials, making it the only trader with a positive profit rate above 90% across all four conditions. In the stable condition CT earns a mean profit of 6,221, just behind ZIP's 7,837. In shock up, CT's position limit contains the inventory accumulation that, as previously discussed, exposes ZIP's short position to the sudden price increase at $t = 1200$, resulting in

Trader Type	**Profit**	σ	**No. of Trades**	**Profit/σ**
ZIP	7,061	4,723	3,750	1.50
CT	5,165	2,064	4,675	2.50
PRSH	1,951	6,229	1,212	0.31
ZIC	1,274	3,547	589	0.36
SNPR	−1,170	781	1,269	−1.50
SHVR	−5,150	2,636	5,855	−1.95
GVWY	−9,020	4,100	6,518	−2.20

Table 3: Full results of the main simulation across 150 trials per condition for all four conditions, with all seven trader types present (5 per type, 35 per session).

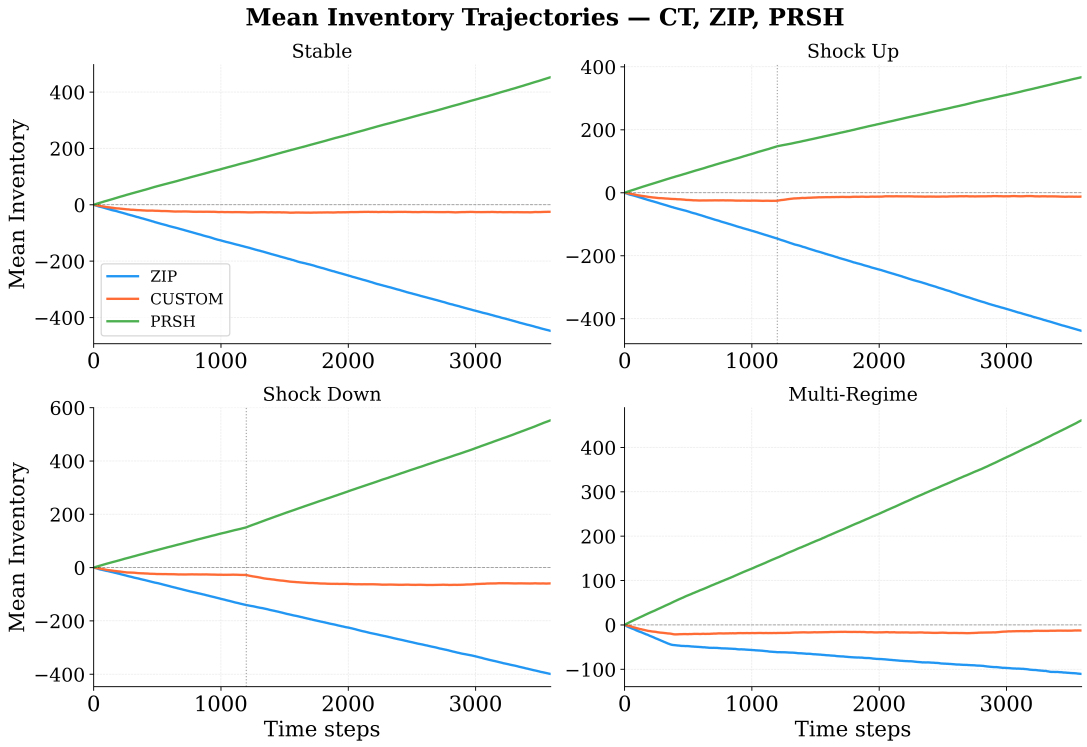

Figure 3: Mean inventory trajectories for CT, ZIP and PRSH across all four market conditions, averaged over 150 trials per condition. The vertical dashed lines in the shock-up and shock-down panels mark $t = 1200$, at which point the price offset is applied.

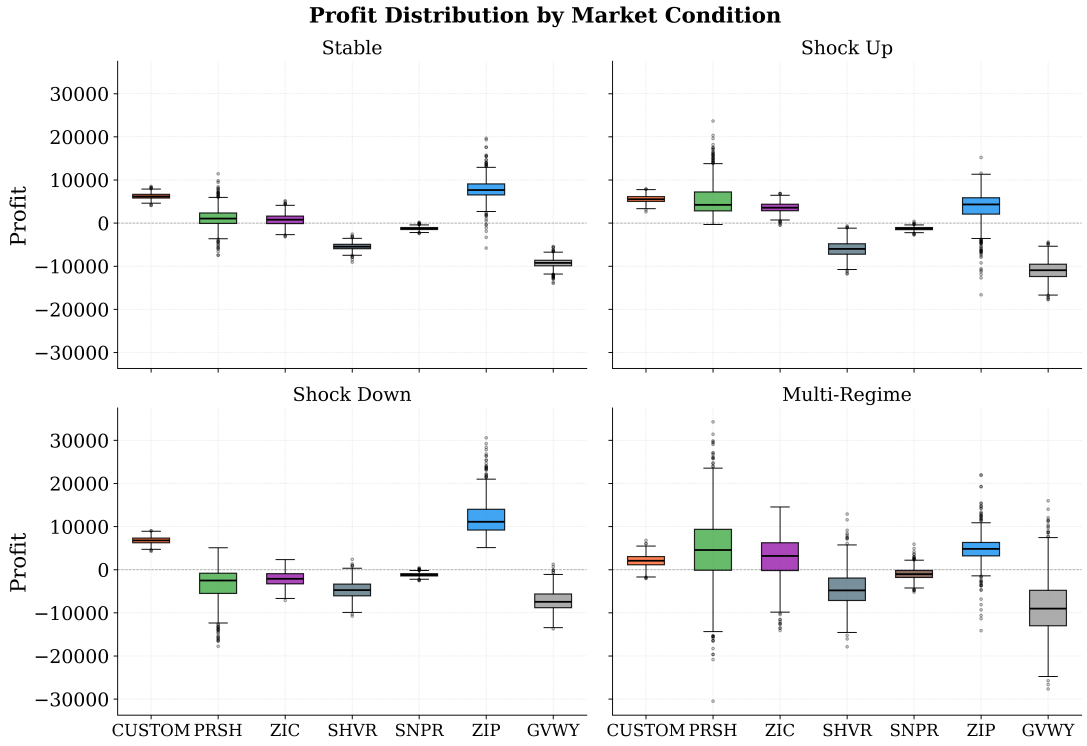

Figure 4: Profit distributions across all four market conditions for all seven trader types, from 150 trials per condition. Each box shows the interquartile range, with whiskers extending to 1.5 times the interquartile range and outliers shown individually.

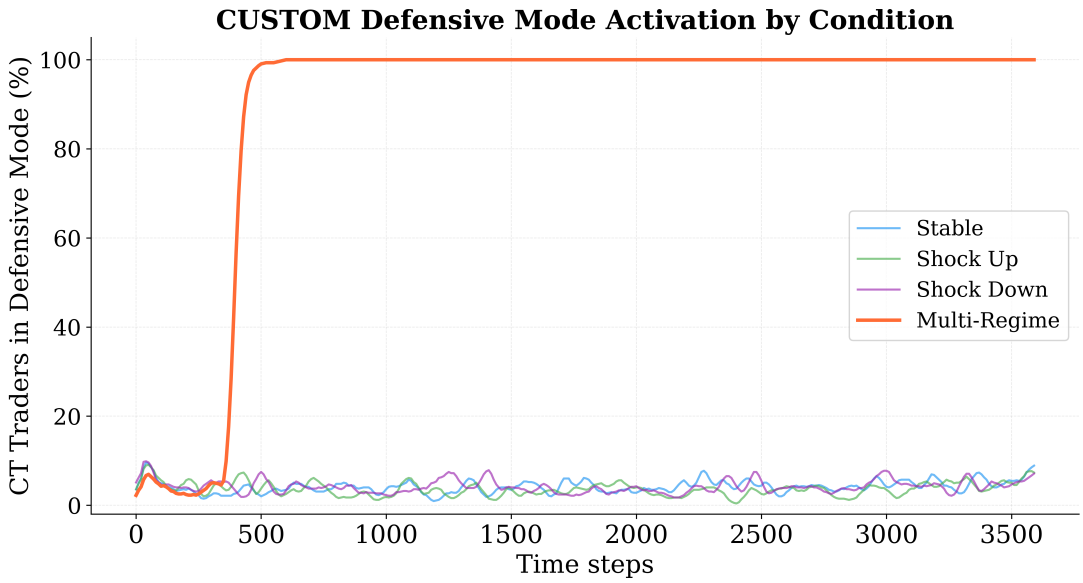

Figure 5: Fraction of CT traders in defensive mode at each time step, averaged across 150 trials per condition. Persistent volatility above the threshold $\sigma_v = 14.99$ in the multi-regime condition triggers and sustains defensive mode from approximately $t = 450$ onwards.

CT earning 5,558 against ZIP's 3,564. PRSH also performs strongly in shock up at 5,532, as its long position benefits directly from the rising price in the same way CT's bounded inventory protects it from the downside. In shock down the scenario takes a complete turn, where ZIP's short position now benefits from the falling price and ZIP leads the field at 12,033. CT earns 6,798, building a modest short through its normal trading behaviour but unable to match ZIP's scale given its position limit. PRSH collapses to $-3{,}503$ in shock down, profitable in only 12.5% of trials, as its persistent long inventory is fully exposed to the falling price in exactly the manner described above.

The multi-regime condition produces CT's weakest result at a mean profit of 2,083, behind ZIP (4,808) and PRSH (4,591). Figure 5 shows its defensive mechanism in action, where it spends 89.2% of the multi-regime session in defensive mode, compared to approximately 4% in each of the other three conditions. The repeated random offset changes in the multi-regime condition drive market volatility above $\sigma_v = 14.99$ early in the session, triggering the mechanism. As discussed in the choice of parameters, this greatly reduced its execution probability in the LOB and therefore the trades from which profit can accumulate. In the stable, shock-up and shock-down conditions volatility does not persistently exceed the threshold, so defensive mode activates only briefly and CT's normal trading strategy applies for the bulk of each session. Despite its multi-regime underperformance, CT maintains a head-to-head win rate of 74.2% against PRSH across all conditions, and 27.1% against ZIP, with its strongest relative results against ZIP occurring in shock up.

Bootstrap Resampling Analysis

Figure 6 shows the bootstrap distributions of mean profit for CT, ZIP and PRSH, using the method described earlier. The three distributions are clearly separated, with no pairwise overlap in their 95% confidence intervals. CT's interval of [4,822, 5,488] lies entirely below ZIP's [6,319, 7,828] and entirely above PRSH's [981, 2,970]. This helped to conclude that the rankings from Table 3 are not completely by chance but instead reflect a consistent property of

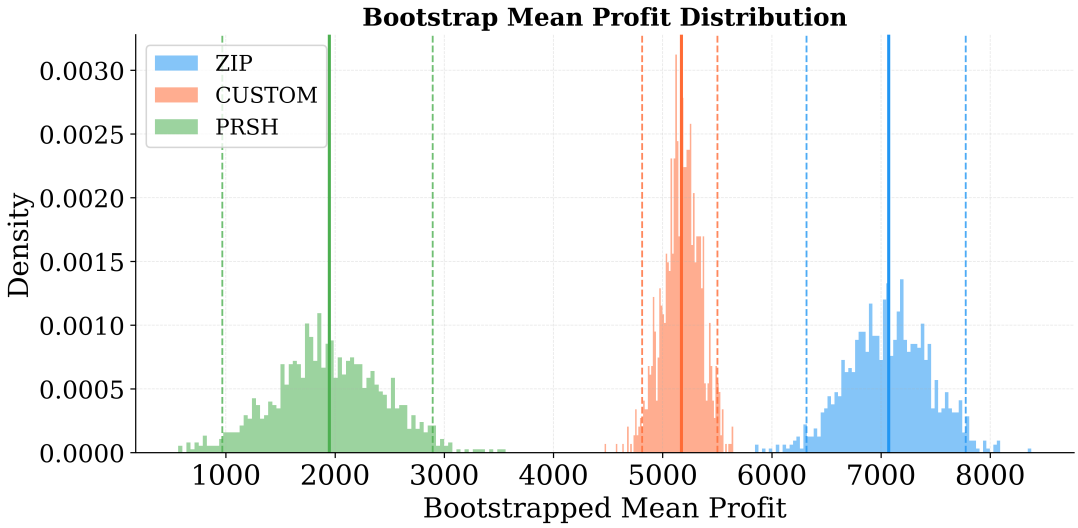

Figure 6: Bootstrap distributions of mean profit for CT, ZIP and PRSH from $K = 10{,}000$ resamples of 150 trials. Solid vertical lines mark the bootstrap mean for each trader, with dashed lines mark the 95% confidence interval bounds.

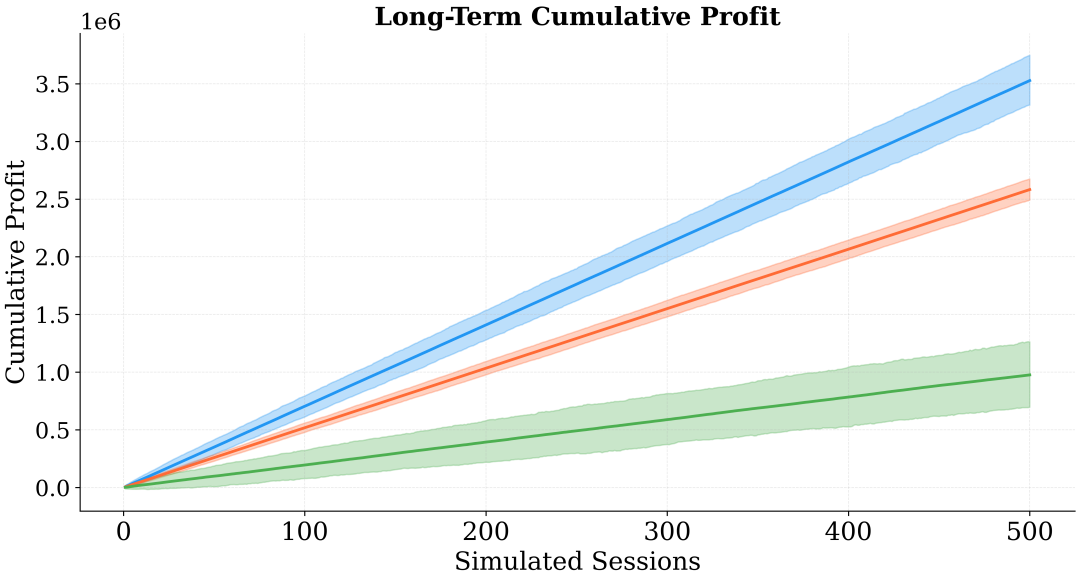

Figure 7: Long-term cumulative profit projected over 500 sessions for CT, ZIP and PRSH from $B = 1{,}000$ bootstrap iterations. Solid lines show the mean cumulative trajectory and shaded bands show the 95% confidence interval.

each trader's strategy across all draws from the observed profit distributions. CT's confidence interval is the narrowest of the three, with a width of 666 compared to ZIP's 1,509 and PRSH's 1,989. This is a direct consequence of CT's lower per-trial profit variance and confirms that its mean estimate carries the least uncertainty of any profitable trader in the field. Figure 7 shows the long-term cumulative profit projected over 500 simulated sessions from $B = 1{,}000$ bootstrap iterations. CT's confidence band is narrower than ZIP's at every horizon and by session 500, CT's projected cumulative profit falls within a substantially tighter range, meaning that over a long sequence of sessions CT's total profit is more predictable. ZIP's higher expected cumulative total is consistent with its higher mean profit per session, but the wider band reflects its larger per-session variance. At session 500, CT's 95% confidence band does not overlap with

PRSH's, confirming that CT's long-run cumulative gains over PRSH is sustained as the number of sessions grows, but falls short in comparison to ZIP.

Discussion

The results confirm that CT's inventory controls achieved their intended purpose. By keeping the short position that its "ZIP-like" learning rule naturally produces within a manageable range, CT remained profitable across a range of price movements, rather than allowing its inventory to diverge as ZIP's does. By making inventory control an explicit mechanism rather than a side effect of quoting behaviour, CT achieved a more consistent profit profile than any other trader in the field, despite finishing second to ZIP by mean profit overall. The shock-up condition is the clearest demonstration of this, where ZIP's unconstrained short position is directly exposed to the sudden price increase at $t = 1200$. CT, on the other hand, has its position limit that contains the damage and allows profitable quoting to continue, resulting in CT beating the other traders with a mean profit of 5,558 against ZIP's 3,564. The multi-regime condition is its weakest performance, and its cause seemed to lie within how the parameter space was constructed, rather than the mechanism on its own being fundamentally weak for an environment such as this one. Once the volatility threshold $\sigma_v = 14.99$ is breached early in a multi-regime session, CT's defensive mode activates and does not deactivate, effectively reducing it to a passive participant for the remainder of that session. Readjusting CT's mechanism to allow market re-entry once volatility falls to a value greater than half the threshold, allowing CT to be more active in volatile sessions, would be a natural and well-motivated extension of this work.

It is also worth noting what happens to ZIP's performance once CT enters the market. In the preliminary study, ZIP's mean profit was 10,053. In the main simulation it drops to 7,061, a fall of nearly 30%. ZIP was originally designed by Cliff and Bruten [5] to explore how little intelligence a trader needs to be competitive in a CDA, and was subsequently shown by Das *et al.* [12] to be capable of outperforming human traders in laboratory exchange experiments. Despite this, ZIP's strength in this environment is not a product of its learning rule being well suited to a setting where inventory management was integrated to assess its importance in profit accumulation. ZIP was designed for a standard CDA with fixed buyer and seller roles, where inventory does not accumulate in a way that directly affects final profit. Here, it happens to build a short position as a structural consequence of its margin-tightening behaviour, and that position happens to be profitable under most price schedules. CT was designed to make exactly this kind of inventory management deliberate, and competes for the same profitable trades as ZIP, which is why ZIP's mean profit falls so sharply when CT is introduced. PRSH, by contrast, is algorithmically more sophisticated than ZIP in the sense that it maintains multiple parallel strategies and uses "hill-climbing" to select among them, but its adaptation frequency of 15 evaluation cycles per session cannot match ZIP's per-trade updates.CT sits between them, taking ZIP's adaptation speed and combining it with explicit inventory controls and volatility detection that neither ZIP nor PRSH possess, while avoiding the slow update cycle that limits PRSH's ability to respond to evolving market conditions.

There were also constraints imposed by the scope of this study that are worth acknowledging separately from the simulations' structural limitations. The GP ran for 100 iterations over a three-dimensional parameter space, where each parameter combination only had 40 trials dictate its performance. In the broader Bayesian optimisation literature, parameter searches of this kind typically operate over many more evaluations and higher-dimensional spaces [8], and the studies that ZIP and PRSH originally appeared in were validated across far larger numbers of independent trials than the 150 per condition used here. As each GP evaluation required a full multi-condition simulation run, scaling to thousands of iterations would have been computa-

tionally excessive for a study of this scope. The boundary collapse at $\sigma_v = 14.99$ is the most concrete consequence of this constraint, as it suggests the true optimum for multi-regime conditions may lie outside the range that was explored, and a wider search could recover some of the multi-regime underperformance.

The Value of the BSE

The BSE is, by design, a minimal simulation. It captures the core mechanism of a real financial exchange through a continuous double auction with an LOB, but strips away most of the complexity that characterises real markets. There is no latency present in the BSE, meaning every trader receives the current state of the LOB instantly and responds without delay. A trader selling 400 shares in a BSE session has no impact on the wider market in the way a large institutional seller would in a real exchange. The trader population is fixed at 35 agents of seven known types, whereas real markets contain thousands of participants with private information and strategies that evolve continuously.

That being said, steps were taken in this study to move closer to a realistic environment than the standard BSE setup allows. The profit measure was redefined to capture the full consequence of inventory accumulation rather than treating each trade as a settled transaction. Role assignment was randomised at each time step, and limit prices were drawn from overlapping ranges either side of the set SP_0 to ensure a well-defined price centre, allowing the strategies within the BSE to still act with their internal structure, such as GVWY only putting through orders at the limit price, or SHVR over/under cutting its limits by one tick, producing the expected outcomes. These changes make the simulation more demanding than the standard BSE setup, since traders can now lose money and inventory management is no longer irrelevant. But even in these efforts, the gap between this and a real exchange remains large. What cannot be claimed is that CT would necessarily outperform ZIP or PRSH in a real market, or vice versa, where the conditions that make its inventory management an advantage might not apply, as the results are an argument for the design approach and not a proof of real-world profitability.

The value of this study lies in using a controllable environment to ask questions that real markets cannot answer. Real markets cannot be paused, replicated, or randomised, whereas the BSE can be. The ability to run 150 independent trials of the same condition and to isolate the effect of individual parameters through Bayesian optimisation gives a level of insight into trader design that analysing real market data simply cannot provide. The central finding, that inventory management rather than learning sophistication drives consistent profitability in a CDA with this profit definition, is a result that emerges from that controlled environment and one that would be very difficult to arrive at through any other means.

The results point toward several concrete improvements that would form the natural next steps in CT's development. The most directly motivated is a redesign of the volatility threshold, by allowing CT re-enter markets when the volatility drops to $\leq 0.8\sigma_v$. Another change also worth doing would be replacing the fixed 50-trade volatility window with an exponential moving average, which weighs recent price changes more heavily than older ones, which would allow CT to respond more fluidly to short-lived spikes without committing to a defensive mode that hinders its ability to profit the way other traders might. This important modification is small in implementation terms but would allow CT to buy or sell shares more appropriately, and address the underperformance in the shock-down and multi-regime markets.

Conclusion

This study set out to build and evaluate a new algorithmic trader in the BSE, designed around the idea that explicitly managing inventory accumulation would produce more consistent profitability across a range of market conditions than strategies that leave inventory as an unmanaged side effect of their internal behaviour. A Custom Trader was designed with three Bayesian optimised parameters controlling its learning rate, position limit, and volatility threshold, and was tested against six established BSE traders across 150 independent trials of four market conditions. CT finished second overall by mean profit at 5,165, behind ZIP at 7,061, and led the field in the shock-up condition outright at 5,558 against ZIP's 3,564. It was the only trader to return a positive profit in more than 90% of trials across all four conditions, and its ratio of mean profit to standard deviation of 2.50 was the highest of any trader in the field. Bootstrap resampling over 10,000 iterations confirmed that the ranking of ZIP above CT above PRSH is not a product of chance across the 150 trials.

What the results ultimately show is that in this market environment, it is not the sophistication of a trader's learning algorithm that determines how reliably it profits, but how well it manages the inventory position that its learning behaviour naturally produces. ZIP, despite its historical reputation and strong overall performance, succeeds largely because its margin-tightening rule happens to build a short position that is profitable under most conditions. CT was designed to make that same process deliberate, and the results show that doing so produces a more resilient strategy in conditions where prices move against a trader's position. The multi-regime underperformance is the clearest limitation of the current design, and it points to a specific and fixable problem in how the volatility threshold operates rather than a weakness in the broader approach.

The most natural extensions of this work are expanding the volatility threshold search range beyond the boundary that the Bayesian optimisation converged to, replacing the fixed volatility window with an exponential moving average to allow more adaptive detection of regime changes, and running a larger number of trials and optimisation iterations to tighten the confidence intervals on all results. Each of these is a modest step in isolation, but together they would address the three main constraints that the scope of this study imposed on what could be concluded. The BSE makes all of this feasible in a way that real financial markets simply do not, and that is precisely where its value as a research tool lies.

Acknowledgements

I would like to thank Professor Dave Cliff of the University of Bristol for developing and openly sharing the Bristol Stock Exchange simulator, without which this project could not have been done.
I would also like to thank Joe Page, a fellow researcher working independently on the BSE, for the many collaborative discussions that inspired the profit redefinition and inventory framework central to this work.

References

[1] BestBrokers, 2025. *Stock market trading statistics 2026: size and value trends* [Online]. Available from: `https://www.bestbrokers.com/stock-trading/stock-trading-market-statistics/` [Accessed 4 May 2026].

[2] Mordor Intelligence, 2026. *Algorithmic trading market size & share trends, 2031* [Online]. Available from: https://www.mordorintelligence.com/industry-reports/algorithmic-trading-market [Accessed 2 April 2026].

[3] Cliff, D., 2012. *BSE: the Bristol Stock Exchange. A minimal simulation of a limit order book financial exchange* (Version 1.2) [Online]. Bristol: University of Bristol. Available from: https://github.com/davecliff/BristolStockExchange [Accessed 1 November 2025].

[4] Gode, D.K. and Sunder, S., 1993. Allocative efficiency of markets with zero-intelligence traders: market as a partial substitute for individual rationality. *Journal of political economy*, 101(1), pp.119–137.

[5] Cliff, D. and Bruten, J., 1997. *Zero is not enough: on the lower limit of agent intelligence for continuous double auction markets*. (Technical Report HPL-97-141). Bristol: Hewlett-Packard Laboratories.

[6] Cliff, D., 2021. Parameterised response zero intelligence traders. *Journal of economic interaction and coordination* [Online], 19(3), pp.439–492. Available from: https://doi.org/10.1007/s11403-023-00388-7 [Accessed 10 November 2025].

[7] Fukuma, N. and Kadogawa, Y., 2020. *An overview of algorithmic trading in foreign exchange markets and its impacts on market liquidity* [Online]. Bank of Japan. Available from: https://www.boj.or.jp/en/research/wps_rev/rev_2020/data/rev20e05.pdf [Accessed 2 April 2026].

[8] Garnett, R., 2023. *Bayesian optimization*. Cambridge: Cambridge University Press.

[9] Jiang, C. and Li, M., 2025. Trading off quality and uncertainty through multi-objective optimisation in batch Bayesian optimisation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(25), pp.27027–27035. Available from: https://doi.org/10.1609/aaai.v39i25.34909 [Accessed 5 February 2026].

[10] Jones, D.R., Schonlau, M. and Welch, W.J., 1998. Efficient global optimization of expensive black-box functions. *Journal of global optimization*, 13(4), pp.455–492.

[11] Head, T., Kumar, M., Nahrstaedt, H., Louppe, G. and Shcherbatyi, I., 2020. *scikit-optimize/scikit-optimize* (v0.8.1) [computer program]. Available from: https://doi.org/10.5281/zenodo.4014775 [Accessed 20 April 2026].

[12] Das, R., Hanson, J.E., Kephart, J.O. and Tesauro, G., 2001. Agent-human interactions in the continuous double auction. In: B. Nebel, ed. *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI'01)*, Seattle, August 2001. San Francisco: Morgan Kaufmann, pp.1169–1176.

[13] Genton, M.G., 2001. Classes of kernels for machine learning: a statistics perspective. *Journal of machine learning research*, 2, pp.299–312.

Appendix

Gaussian Process: Mathematical Derivation

The Gaussian Process used in the Bayesian optimisation is characterised by a Matérn covariance kernel [13]. The role of the kernel is to measure how similar two parameter vectors $\mathbf{x}_1$ and $\mathbf{x}_2$ are, and therefore, in this context, how much the profit observed at one combination tells us about the likely profit at another. Two vectors that are close together in parameter space are expected to produce similar profits; two vectors that are far apart carry less information about each other. The kernel formalises this idea:

$$k(\mathbf{x}_1, \mathbf{x}_2) = \frac{1}{\Gamma(\nu)\, 2^{\nu-1}} \left(\frac{\sqrt{2\nu}\, d}{l} \right)^{\nu} K_\nu \left(\frac{\sqrt{2\nu}\, d}{l} \right) \tag{17}$$

where $d = \|\mathbf{x}_1 - \mathbf{x}_2\|$ is the distance between the two parameter vectors, l is a length-scale controlling how quickly the kernel decays with distance (i.e. how far apart two vectors can be while still being considered informative about each other), ν controls the assumed smoothness of the profit function, K_ν is the modified Bessel function of the second kind, and Γ is the gamma function.

Suppose the GP has already evaluated N parameter vectors, collected into a matrix $\mathbf{X} \in \mathbb{R}^{N\times 3}$, with corresponding observed mean profits stored in a vector $\mathbf{y} \in \mathbb{R}^N$. To predict the profit at a new, unevaluated parameter vector $\mathbf{x}^*$, the GP computes two quantities.

The first is $\mathbf{k}^* \in \mathbb{R}^N$, a vector of kernel evaluations between $\mathbf{x}^*$ and each of the N already-evaluated vectors:

$$\mathbf{k}^* = \begin{pmatrix} k(\mathbf{x}^*, \mathbf{x}_1) \\ k(\mathbf{x}^*, \mathbf{x}_2) \\ \vdots \\ k(\mathbf{x}^*, \mathbf{x}_N) \end{pmatrix} \tag{18}$$

Each entry quantifies how similar $\mathbf{x}^*$ is to one of the previously evaluated vectors. The second is $\mathbf{K} \in \mathbb{R}^{N\times N}$, the kernel matrix of all pairwise similarities between the already-evaluated vectors:

$$K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j) \tag{19}$$

Using these, the GP posterior at $\mathbf{x}^*$ is a normal distribution with mean and variance given by:

$$\mu(\mathbf{x}^*) = \mathbf{k}^{*\top} \left[\mathbf{K} + \sigma_n^2 \mathbf{I} \right]^{-1} \mathbf{y} \tag{20}$$

$$\sigma^2(\mathbf{x}^*) = k(\mathbf{x}^*, \mathbf{x}^*) - \mathbf{k}^{*\top} \left[\mathbf{K} + \sigma_n^2 \mathbf{I} \right]^{-1} \mathbf{k}^* \tag{21}$$

where σ_n^2 is the noise variance on the profit observations and $\mathbf{I}$ is the $N \times N$ identity matrix. The term $\left[\mathbf{K} + \sigma_n^2 \mathbf{I} \right]^{-1} \mathbf{y}$ can be thought of as a set of weights learned from the observed profits; $\mu(\mathbf{x}^*)$ is then a weighted sum of how similar $\mathbf{x}^*$ is to each observed point, giving a best estimate of the profit there. The variance $\sigma^2(\mathbf{x}^*)$ measures how uncertain that estimate is: parameter vectors close to previously evaluated points will have low uncertainty, while those in unexplored regions of the space will have high uncertainty.

Bayesian Optimisation: Python Implementation

The Bayesian optimisation was implemented in Python using the Scikit-Optimize library [11], which provides the `gp_minimize` function. The objective function receives a parameter combination and returns the negative mean profit of CT across a short simulation run (negated because `gp_minimize` minimises by convention).

```python
def objective(params):
    alpha, position_limit, volatility_threshold = params
    custom_params = {
        'alpha': alpha,
        'position_limit': int(position_limit),
        'volatility_threshold': volatility_threshold,
        'initial_margin': 0.05,
        'defensive_margin': 0.15
    }
    result = run_simulation(custom_params)
    mean_profit = result['custom_trader_mean_profit']
    return -mean_profit  # minimise negative profit
```

Listing 1: Objective function passed to the GP

```python
from skopt import gp_minimize
from skopt.space import Real, Integer

search_space = [
    Real(0.001, 0.30,  name='alpha'),
    Integer(20,  800, name='position_limit'),
    Real(3.0,  15.0, name='volatility_threshold')]

result = gp_minimize(
    objective,
    search_space,
    n_calls=100,          # Total evaluations
    n_initial_points=20, # Random evaluations before GP takes over
    acq_func='EI',       # Expected Improvement acquisition function
    random_state=1)
```

Listing 2: GP minimisation call with parameter bounds

The 20 initial random evaluations seed the GP with enough coverage of the parameter space to build a meaningful prior. The subsequent 80 evaluations are guided by Expected Improvement, which at each step identifies the combination most likely to exceed the current best observed profit. On completion, `result.x` contains the optimal parameter values used for all main simulation runs.